TP3: Gérer le conteneurs avec podman (Préparation Certification Red hat specialist in containers)

1. pour préparer l'environnement de simulation fait dans une vm redhat:

```
git clone https://github.com/halekammoun/Guide-complet-sur-la-conteneurisation.git
```

```
cd Guide-complet-sur-la-conteneurisation/TPs
```

```
bash prep.sh
```

2. pour lancer la simulation fait: https://ex188-validation-test.lovable.app/
3. pour avoir un score

```
bash grading.sh
```

Enoncé

task 1

```
1. Running Simple Containers
ACME Corporation wants to demonstrate a simple microservice application using containers. As an
initial demonstration, deploy an Nginx container.

Start a container that meets the following requirements:
Use the image docker.io/library/nginx:latest
The container must be named acme-demo-html
The container must run in detached mode
Expose container port 80 on host port 8001
Map the host directory $HOME/workspace1/acme-nginx-html to the container path /usr/share/nginx/html
The website must return: Hello World NginX
Verification
curl localhost:8001 -> Hello World NginX
```

task 2

```
2. Interacting with Containers
ACME Corporation wants to demonstrate how administrators can modify and reload a running container.

Start a container that meets the following requirements:
Use the image docker.io/library/nginx:latest
The container must be named acme-demo-nginx
Run the container in detached mode
```

```
Expose container port 80 on host port 8002
Copy $HOME/workspace2/acme-nginx-web/html into the container path /usr/share/nginx/html
Copy $HOME/workspace2/acme-nginx-web/conf/nginx.conf into the container path
/etc/nginx/conf.d/default.conf
Reload Nginx after modifying the configuration
Verification
curl localhost:8002
```

task 3

```
3. Injecting Variables into Containers
ACME Corporation wants to demonstrate how environment variables can be injected into containers.

Use the image quay.io/myacme/welcome
Create two containers named acme_nginx_container_1 and acme_nginx_container_2
The first container must use RESPONSE="Welcome_ACME_Nginx_Container_1"
The second container must use RESPONSE="Welcome_ACME_Nginx_Container_2"
Both containers must run in detached mode
Container port 8080 must be mapped to host port 8003
Both containers must coexist, but only one may run at a time because they use the same host port
Verification
curl localhost:8003
```

task 4

```
4. Building Custom Container Images
Task 4.1 – Custom MariaDB Image
Build a custom image using the initial Containerfile located at $HOME/workspace4/acme-mariadb-
containerfile

Use docker.io/library/mariadb:latest
Define the build arguments ACME_MARIADB_DATABASE and ACME_MARIADB_PASSWORD
Define the environment variables MARIADB_DATABASE and MARIADB_ROOT_PASSWORD
Map the build arguments to the environment variables
Build using ACME_MARIADB_DATABASE=acme and ACME_MARIADB_PASSWORD=acme
Tag the image acme:5000/acme-mariadb:latest
Task 4.2 – MariaDB Export Image
Use $HOME/workspace4/acme-mariadb-db/acme-db-export-containerfile

Use the official MariaDB image
Install the MariaDB client tools required for mariadb-dump
Set the working directory to /scripts
Copy scripts/export.sh from the host to /scripts/export.sh in the container
Make export.sh executable
Configure the image to execute the script automatically
Build and tag the image as acme-mariadb-export:latest
```

task 5

```
5. Multi-Container Deployment
```

```
Create the following network and volumes:
Create the network acme-wp-net
Create the volumes acme-wp-backend, acme-wp-app and acme_wordpress_data
MariaDB Container
Image: quay.io/myacme/mariadb:latest
Name: mariadb
Network: acme-wp-net
Volume: acme-wp-backend:/bitnami/mariadb
Environment variables: MARIADB_USER=acme_wordpress, MARIADB_PASSWORD=acme,
MARIADB_DATABASE=acme_wordpress, MARIADB_ROOT_PASSWORD=acme
Nginx Container
Image: quay.io/myacme/nginx
Name: acme-wp-app
Network: acme-wp-net
Volume: acme-wp-app:/etc/nginx
Port mapping: 8080:8080
WordPress Container
Image: quay.io/myacme/wordpress:latest
Name: acme-wordpress
Network: acme-wp-net
Volume: acme_wordpress_data:/bitnami/wordpress
Port mappings: 8004:8080 and 8443:8443
Environment variables: WORDPRESS_DATABASE_USER=acme_wordpress, WORDPRESS_DATABASE_PASSWORD=acme,
WORDPRESS_DATABASE_NAME=acme_wordpress
All containers must run persistently
```

task 6

```
6. Troubleshooting Containerized Applications
ACME Corporation has deployed a containerized WordPress application consisting of a MariaDB
database, a WordPress PHP application and an Nginx reverse proxy. The application is currently
experiencing configuration and connectivity problems. Your task is to deploy the required
infrastructure, investigate the problems, correct the configuration, and make the WordPress website
available.

Create the network sks-wp-ts
Create the volumes sks-backend-ts-vol and sks-wp-app-ts-vol
All containers must be attached to sks-wp-ts
Mount sks-wp-app-ts-vol:/var/www/html for both the WordPress application and frontend containers
Mount sks-backend-ts-vol:/var/lib/mysql for the database container
Only the frontend container must publish a host port
The frontend container must use 8084:80
The WordPress website must be accessible at http://localhost:8084
Required Containers
Container Name  Purpose Image
sks-backend-ts  Database    docker.io/skstechnologia/wp-mariadb:latest
sks-wp-app-ts   WordPress   docker.io/skstechnologia/wp-app:ts
sks-frontend-ts Reverse Proxy   docker.io/skstechnologia/wp-frontend:ts
```

Correction

task 1

```
podman run -d --name acme-demo-html -p 8001:80 -v $HOME/workspace1/acme-nginx-
html:/usr/share/nginx/html:Z docker.io/library/nginx:latest
```

sans :Z ça nous donne l'erreur Forbidden 403

```
podman ps
```

Tester Port Mapping

```
curl localhost:8001
```

```
podman exec <idcontainer> curl localhost:80
```

ou bien entrer avec shell puis faire le curl

```
podman exec -it <idcontainer> bash
```

```
curl localhost:80
```

task 2

```
podman run -d --name acme-demo-nginx -p 8002:80 docker.io/library/nginx:latest
```

```
podman cp $HOME/workspace2/acme-nginx-web/html/. <idcontainer>:/usr/share/nginx/html

ou

podman cp $HOME/workspace2/acme-nginx-web/html/index.html <idcontainer>:/usr/share/nginx/html

ou

podman cp $HOME/workspace2/acme-nginx-web/html/* <idcontainer>:/usr/share/nginx/html
```

si vous oubliez de copier le contenu et pas le répertoire nginx ne trouve pas notre page web et affiche la page web par défaut le path sera: `/usr/share/nginx/html/html/index.html` qui est supposé etre `/usr/share/nginx/html/index.html`

```
podman cp $HOME/workspace2/acme-nginx-web/conf/nginx.conf
<idcontainer>:/etc/nginx/conf.d/default.conf
```

```
podman exec <idcontainer> nginx -s reload
ou
podman exec -it <idcontainer> bash
nginx -s reload
```

task 3

Le port de l'hôte (host port) ne peut pas être associé simultanément à deux services différents sur la même adresse IP (notre host). Or, les deux conteneurs utilisent le même mapping -p 8003:8080, ce qui crée un conflit, car le port 8003 de l'hôte est déjà utilisé par le premier conteneur. Il faut donc le premier conteneur pour créer le second.

```
podman run -d --name acme_nginx_container_1 -e RESPONSE="Welcome_ACME_Nginx_Container_1" -p
8080:8003 quay.io/myacme/welcome
```

```
curl localhost:8003
```

ça donne `Welcome_ACME_Nginx_Container_1`

```
podman stop acme_nginx_container_1
```

```
podman run -d --name acme_nginx_container_2 -e RESPONSE="Welcome_ACME_Nginx_Container_2" -p
8080:8003 quay.io/myacme/welcome
```

ça donne `Welcome_ACME_Nginx_Container_2`

task 4

```
vim $HOME/workspace4/acme-mariadb-containerfile
```

```
FROM docker.io/library/mariadb:latest
ARG ACME_MARIADB_DATABASE
ARG ACME_MARIADB_PASSWORD
ENV MARIADB_DATABASE=$ACME_MARIADB_DATABASE
ENV MARIADB_ROOT_PASSWORD=$ACME_MARIADB_PASSWORD
```

```
podman build -t acme:5000/acme-mariadb:latest --build-arg ACME_MARIADB_DATABASE=acme --build-arg
ACME_MARIADB_PASSWORD=acme -f $HOME/workspace4/acme-mariadb-containerfile
ou
podman build -t acme:5000/acme-mariadb:latest --build-arg ACME_MARIADB_DATABASE=acme --build-arg
ACME_MARIADB_PASSWORD=acme -f acme-mariadb-containerfile $HOME/workspace4/
```

task 5

```
podman network create acme-wp-net
```

```
podman volume create acme-wp-backend
```

```
podman volume create acme-wp-app
```

```
podman volume create acme_wordpress_data
```

```
podman run -d --name mariadb --network acme-wp-net -v acme-wp-backend:/bitnami/mariadb:Z -e
MARIADB_USER=acme_wordpress -e MARIADB_PASSWORD=acme -e MARIADB_DATABASE=acme_wordpress -e
MARIADB_ROOT_PASSWORD=acme quay.io/myacme/mariadb:latest
```

```
podman run -d --name acme-wp-app --network acme-wp-net -v acme-wp-app:/etc/nginx:Z -p 8080:8080
quay.io/myacme/nginx
```

```
podman run -d --name acme-wordpress --network acme-wp-net -v
acme_wordpress_data:/bitnami/wordpress:Z -p 8004:8080 -p 8443:8443 -e
WORDPRESS_DATABASE_USER=acme_wordpress -e WORDPRESS_DATABASE_PASSWORD=acme -e
WORDPRESS_DATABASE_NAME=acme_wordpress quay.io/myacme/wordpress:latest
```

```
curl http://localhost:8004
```

task 6

Consulter la correction dans le PDF